Plateforme de marché financier simulé

1. Objectif métier et périmètre

Une équipe de suivi de marché souhaite observer les échanges de vingt actions, comprendre les tendances et repérer les variations importantes. La plateforme transforme des transactions individuelles en informations utilisables : chandeliers, moyennes mobiles, rendements, volatilité, VWAP, corrélations et alertes. Elle fournit un support à l'analyse, sans prétendre prédire les cours.

Deux besoins justifient deux chaînes indépendantes. Le suivi continu exige des bougies rapidement accessibles dans Grafana. L'analyse historique exige une archive durable, des transformations rejouables et des métriques cohérentes dans Streamlit. Les deux utilisent le même générateur, le même contrat et des dates UTC.

Les symboles correspondent à des entreprises réelles, mais les prix et transactions sont simulés. La démonstration ne constitue pas une source de cotations réelles. Le générateur produit un choc contrôlé sur AAPL pour rendre observable un mouvement inhabituel.

Cette version applique les précisions du 4 novembre 2026 : Gold calculé par Spark SQL et stocké dans Iceberg ; alertes exclusivement dans PostgreSQL ; aucun catalogue Hive. Les intervalles et seuils du cahier initial sont conservés. Les membres du groupe restent à renseigner dans le README.

2. Diagnostic et adaptation de l'existant

L'analyse a porté sur le cahier Word complet, les services Compose, le générateur, les archives MinIO, Spark, PostgreSQL, Grafana, Dagster et Streamlit. Le tableau détaillé se trouve dans docs/DIAGNOSTIC_V2.md. Les consignes explicites récentes prévalent sur le schéma générique et l'ancienne architecture.

Le dépôt comportait déjà un Producer, Kafka à six partitions, un consommateur Bronze avec manifestes, des calculs OHLCV, PostgreSQL et un tableau Grafana pour trois actions. Ces composants ont été conservés. L'ancienne analytique dbt/DuckDB et le précédent catalogue ne correspondaient plus à l'organisation demandée.

La chaîne continue devient Producer, Kafka, Spark Structured Streaming, PostgreSQL puis Grafana. Le consommateur Bronze conserve son chemin Producer, Kafka, MinIO. Un moteur Spark batch séparé lit ces archives et écrit Silver puis Gold dans Iceberg. Dagster évalue les métriques publiées et conserve les alertes dans PostgreSQL.

Les volumes existants, les fichiers bruts et les anciennes alertes sont préservés. Les nouveaux préfixes Iceberg et checkpoints sont versionnés pour éviter une réutilisation incompatible des journaux. Les documents de la première version sont archivés séparément. La migration est additive ; aucune remise à zéro des données n'est nécessaire.

3. Producer et transport Kafka

Le contrat contient trade_id, symbol, price, quantity et event_time. Les anciens événements portant volume sont acceptés comme alias de quantity. Le prix utilise un décimal et l'horodatage est interprété en UTC. Le symbole doit appartenir aux vingt actifs connus, avec un prix et une quantité strictement positifs.

Le Producer existant est conservé. La clé Kafka est le symbole et le topic trades possède six partitions. Les confirmations de livraison permettent de mesurer le débit effectif. Une configuration de génération à 220 événements par seconde ne suffit pas à prouver que Kafka reçoit ce débit : un échantillonnage réel le vérifie.

Le contrôle exécuté a reçu 5 513 événements en environ 25 secondes, soit 220,25 événements par seconde. Les vingt symboles et les six partitions étaient présents ; chaque clé correspondait au symbole du message. Cette mesure décrit le fonctionnement de la maquette au moment du contrôle.

Kafka permet de découpler les consommateurs. Une interruption du streaming n'arrête pas l'archivage Bronze, et un traitement analytique lit MinIO sans consommer Kafka. La maquette utilise un seul broker : elle montre la séparation des traitements, mais ne constitue pas une infrastructure à haute disponibilité.

4. Calcul des chandeliers en streaming

Spark Structured Streaming consomme Kafka, vérifie le contrat et isole les événements invalides. La déduplication repose sur trade_id avec une watermark de deux minutes. Les données validées et les checkpoints sont conservés sur un volume distinct des fichiers analytiques.

Les bougies ont un intervalle d'une minute. Open et Close correspondent respectivement au premier et au dernier événement selon le couple event_time et trade_id. High et Low sont les extrêmes du prix. Volume est la somme des quantités. Le notional représente la somme prix multiplié par quantité. Le départage par identifiant rend le résultat déterministe lorsque deux horodatages sont identiques.

Deux sorties PostgreSQL servent des finalités différentes : une bougie provisoire se met à jour pendant la minute ; une bougie finalisée est publiée après progression de la watermark. La couche de service empêche une mise à jour provisoire de remplacer une ligne déjà finalisée.

Un test Spark réel a vérifié une bougie de référence : Open 100, High 103, Low 98, Close 103, Volume 10 et notional 1007. Il a aussi contrôlé les doublons, les événements tardifs admis, les événements trop tardifs et la reprise depuis les checkpoints. Le streaming ne dépend plus d'Iceberg.

5. PostgreSQL et Grafana

La table candles possède une clé unique par symbole et début de minute. Les écritures remplacent les agrégats absolus au lieu d'ajouter une seconde fois les volumes. Le rejeu d'un micro-lot ne double donc pas les résultats. Des contraintes vérifient la cohérence OHLC et la positivité des volumes.

La table latest_prices conserve le dernier événement connu. Les opérations de service utilisent une transaction et un verrou PostgreSQL pour éviter les courses entre les sorties provisoires et finalisées. Le test PostgreSQL isolé vérifie le rejeu ainsi que la protection d'une bougie finale.

Grafana conserve le dashboard existant : trois panneaux candlestick pour AAPL, MSFT et NVDA, chacun alimenté par PostgreSQL. Le rafraîchissement de cinq secondes est une fréquence de consultation ; les bougies restent d'une minute. Il ne faut pas confondre ces deux intervalles.

Le test navigateur a réellement ouvert le tableau, vérifié les trois panneaux et l'absence de message No data ou Query error. Les captures de la version actuelle sont dans artifacts/ui. Les accès locaux et les variables de connexion sont documentés dans le README. PostgreSQL héberge aussi le catalogue JDBC et les alertes, dans des structures séparées de la même base.

6. Bronze MinIO préservé

Le consommateur Bronze existant archive les messages Kafka sous forme JSONL et Parquet. Les coordonnées de provenance comprennent le topic, la partition et l'offset. Les messages malformés restent archivés avec une indication d'erreur, ce qui conserve la preuve brute et évite qu'un message inutilisable bloque la consommation.

L'organisation des préfixes json/trades, parquet/trades et manifests/trades est conservée. Un manifeste identifie les objets complets, leur nombre de lignes et les offsets à reprendre. Le consommateur confirme les offsets après la réussite des écritures. Un objet Parquet non accompagné d'un manifeste complet n'entre pas dans le traitement Silver.

Spark batch dresse l'inventaire des manifestes et le compare au registre PostgreSQL lakehouse_files. La clé et l'ETag permettent de sélectionner les fichiers nouveaux ou modifiés. BRONZE_FILES_PER_RUN limite la taille du lot à 200 fichiers par défaut, afin de reprendre progressivement un historique important.

Le test réel a relu un Parquet de 6 660 lignes et confirmé le nombre indiqué dans son manifeste. Les anciens objets et les nouvelles archives sont conservés. Le moteur analytique utilise exclusivement ces fichiers comme source ; il ne dépend ni des tables de streaming PostgreSQL ni d'un export de ce streaming.

7. Catalogue Iceberg sans Hive

Les données Silver et Gold ainsi que leurs fichiers de métadonnées sont stockés dans MinIO. Le catalogue Iceberg JDBC conserve les pointeurs transactionnels dans le schéma iceberg_catalog de la base PostgreSQL finance existante. Aucun nouveau service de base de données n'est ajouté.

Ce choix évite de fonder les commits d'un catalogue Hadoop sur des renommages d'objets S3 et fournit un mécanisme de commit transactionnel adapté au stockage MinIO. La configuration utilise explicitement JdbcCatalog et S3FileIO. Les lectures des Parquet Bronze utilisent S3A. Aucun HiveCatalog ni service Hive n'est présent dans l'architecture active.

Les versions vérifiées sont Spark 3.5.5, Scala 2.12, Iceberg 1.7.2, Java 11 et Hadoop 3.3.4. Le connecteur Hadoop AWS est aligné sur Hadoop 3.3.4. Les dépendances sont résolues pendant la construction de l'image pour rendre les démarrages reproductibles.

Les tables sont placées sous des préfixes lakehouse-v2, sans supprimer l'ancien stockage. Un nouveau processus Spark a pu les retrouver après arrêt du moteur analytique et relire leurs snapshots. Référence technique : https://iceberg.apache.org/docs/1.7.2/jdbc/ ; les détails de configuration figurent dans spark/catalog.py.

8. Silver : transactions canoniques et bougies

La table silver.trades contient les transactions valides et leurs coordonnées Kafka. Le batch normalise les types et accepte l'alias historique volume. Les événements incomplets, les symboles inconnus et les valeurs invalides sont dirigés vers silver.rejected. L'identifiant de transaction constitue la clé de déduplication.

Le contrat considère trade_id comme immuable. Lorsqu'un identifiant existe déjà, sa première version canonique est conservée. Une modification du contenu sous le même identifiant n'est donc pas interprétée comme une correction métier. Cette limite est explicitée dans la documentation.

Le batch identifie les minutes touchées et relit leurs transactions canoniques afin de recalculer entièrement chaque bougie concernée. Les règles Open, High, Low, Close et Volume sont identiques à celles du streaming. Les écritures MERGE remplacent les bougies, sans addition susceptible de doubler les volumes après reprise.

Des contrôles vérifient les valeurs OHLC et l'unicité des clés avant écriture. Les fichiers ne sont marqués comme traités qu'après réussite de Silver. Les minutes modifiées sont inscrites dans lakehouse_dirty afin de déclencher Gold. Le contrôle indépendant a compté 3 576 648 transactions et 5 860 bougies, puis recalculé une bougie depuis les transactions pour comparer tous ses agrégats.

9. Gold et métriques Spark SQL

Gold conserve trois familles de résultats : les métriques minute, les statistiques quotidiennes et les corrélations. Le calcul utilise Spark SQL sur Silver. Les tables sont Iceberg et restent dans MinIO. Les alertes ne sont jamais matérialisées dans une table Iceberg.

Les métriques minute comprennent la clôture, MA20, MA50, le log-rendement, le VWAP cumulé depuis minuit UTC et la variation sur cinq minutes. Les moyennes mobiles utilisent vingt ou cinquante clôtures minute observées. Sans historique suffisant, la valeur reste NULL. La référence de variation doit correspondre à la minute exactement cinq minutes auparavant.

Le VWAP est la somme des notionals divisée par la somme des quantités ; ce n'est pas une moyenne simple des prix. Gold quotidien calcule le rendement entre jours consécutifs, la volatilité des log-rendements minute, le VWAP et la couverture. Les corrélations Pearson utilisent les rendements des actifs sur les minutes communes et des statistiques intermédiaires par jour.

Le traitement reprend les journées touchées et le contexte nécessaire aux fenêtres. Il ne reconstruit pas systématiquement tout l'historique brut. Une publication PostgreSQL expose un ensemble cohérent d'identifiants de snapshots après réussite. Streamlit lit ces snapshots validés. Un test Spark sur des valeurs connues a confirmé MA20 149,5, MA50 134,5 et VWAP 129,5.

10. Alertes relationnelles et historique

L'architecture des alertes est Gold, Dagster, PostgreSQL puis Streamlit. La règle du cahier est conservée : variation absolue strictement supérieure à cinq pour cent sur cinq minutes. Une variation exactement égale à cinq pour cent ne déclenche pas d'alerte. Aucun seuil financier supplémentaire n'est inventé.

Dagster évalue les observations Gold nouvelles ou corrigées, et non uniquement la dernière bougie. Le curseur porte sur la date de modification des métriques pour prendre en compte une correction ancienne. Le traitement ne recalcule pas les métriques financières déjà présentes dans Gold.

Une séquence continue au-dessus du seuil forme un épisode, identifié par un UUID déterministe. Une nouvelle observation dans cet épisode actualise la même alerte. Le retour d'une observation valide dans le seuil la résout. Une référence manquante ne clôture pas artificiellement l'épisode. La clôture est manuelle après résolution, car le cahier ne précise pas cette règle de cycle de vie.

PostgreSQL conserve alerts, alert_observations, alert_history et alert_state. Les contraintes, empreintes et transactions évitent les doublons et conservent les changements de statut. Les 35 anciennes détections sont préservées et identifiées comme historiques. Les tests PostgreSQL couvrent les rejeux, les corrections, la résolution et la clôture idempotente.

11. Dagster, reprise et consultation

Les services continus sont lancés par Docker Compose. Dagster les supervise au moyen d'assets bornés qui vérifient Kafka, la disponibilité des requêtes Spark et les données PostgreSQL. Le job streaming_supervision se termine normalement ; aucune boucle infinie ne monopolise un run.

Le job lakehouse_refresh enchaîne Bronze, Silver, Gold puis alertes. Son schedule actif utilise le cron */5 * * * * en UTC. Un job alerts_refresh permet de réévaluer Gold sans recalcul analytique. Un schedule de fin de journée à 00:07 UTC réutilise le même traitement. La fréquence d'évaluation des alertes ne change pas le grain minute des données.

La file Dagster limite la concurrence. Les writers disposent aussi de verrous PostgreSQL et d'un registre des demandes exécutées. Deux nouvelles tentatives du même identifiant réutilisent le résultat validé. Les étapes possèdent une politique de retry et les runs interrompus restent traçables. Les volumes conservent l'historique d'orchestration et les checkpoints.

Streamlit lit les métriques via le service Spark interne, puis les alertes directement dans PostgreSQL. Les lecteurs utilisent des snapshots Gold publiés et peuvent consulter les résultats pendant un nouveau batch. Les tables actives, l'historique et les changements d'état sont visibles. Le lecteur n'a pas besoin d'une socket Docker ni d'un accès au catalogue Hive.

12. Validation, limites et démonstration

Les validations de cette version ont été exécutées le 4 novembre 2026. Les neuf tests unitaires passent dans le conteneur. Les tests Spark réels couvrent OHLCV, retards, doublons, checkpoints et indicateurs sur des données connues. Les tests PostgreSQL isolés vérifient les upserts et les épisodes d'alerte.

Le run Dagster d5a12d14-9789-4b01-be0d-fcab377bc733 a terminé Bronze, Silver, Gold et alertes. Le run b1eff61b-c4b9-4a55-8889-b27b4f8f0e2a a ensuite réussi avec le service streaming volontairement arrêté. Le cas inverse a également été observé : le streaming a continué pendant l'arrêt du moteur analytique et la relecture indépendante d'Iceberg.

Le navigateur de test a ouvert Streamlit et Grafana, affiché les métriques et alertes et vérifié les trois panneaux candlestick. Un premier délai de chargement Grafana a été dépassé ; le contrôle a réussi avec un délai adapté. L'incompatibilité initiale de MERGE Spark a été corrigée puis validée par les exécutions suivantes. Les erreurs restent dans l'historique.

Les résultats détaillés et les commandes sont dans VALIDATION.md. La maquette n'offre pas de haute disponibilité ni de transaction distribuée entre PostgreSQL et Iceberg. La publication par snapshots et les écritures idempotentes encadrent cette limite. La CI est configurée mais aucune exécution GitHub distante n'est revendiquée. Le guide docs/DEMO.md propose une présentation de sept minutes.